

K-디지털 트레이닝

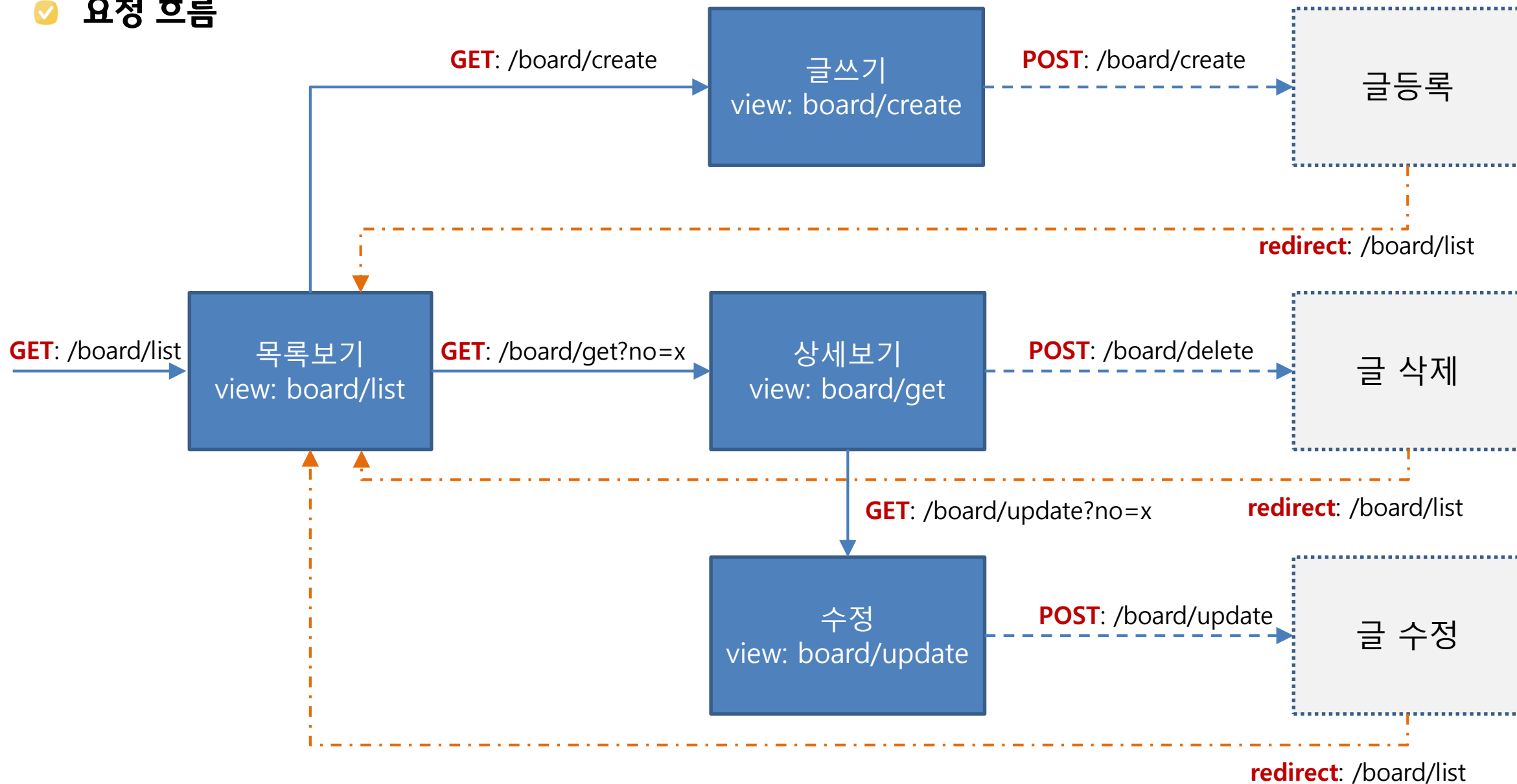
프레젠테이션(웹) 계층의 CRUD 구현

[KB] IT's Your Life

✔ BoardController의 분석

Task	URL	Method	Parameter	Form	URL 이동
전체 목록	/board/list	GET			
등록 처리	/board/create	POST	모든 항목	입력화면 필요	이동
조회	/board/get	GET	no=123		
수정 처리	/board/update	POST	no	입력화면 필요	이동
삭제 처리	/board/delete	POST	모든 항목	입력화면 필요	이동

✓ 요청 흐름



BoardController.java

```
package org.scoula.board.controller;

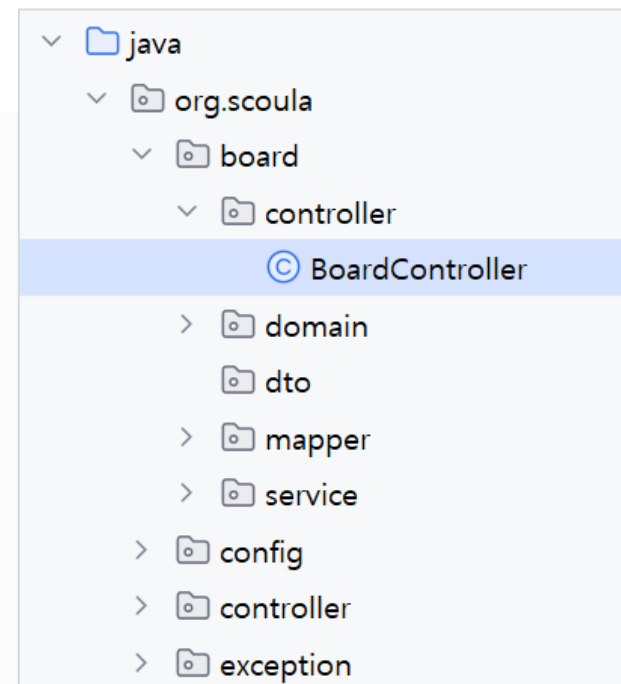
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

import lombok.extern.log4j.Log4j2;

@Controller
@RequestMapping("/board")
@RequiredArgsConstructor
public class BoardController {

    final private BoardService service;

}
```



ServletConfig.java

```
...  
  
@EnableWebMvc  
@ComponentScan(basePackages = {  
    "org.scoula.exception",  
    "org.scoula.controller",  
    "org.scoula.board.controller"  
})  
public class ServletConfig implements WebMvcConfigurer {  
    ...  
}
```

✓ 목록 요청

○ GET 요청

- url: /board/list
- Model 구성
 - 속성명 : list
 - 속성값: service의 getList()로 목록
- View 이름: board/list

BoardController.java

```
...
@RequestMapping("/board")
public class BoardController {
    ...

    @GetMapping("/list") GET :: /board/list
    public void list(Model model) {

        log.info("list");
        model.addAttribute("list", service.getList());

    }
}
```

✓ Jsp에 출력

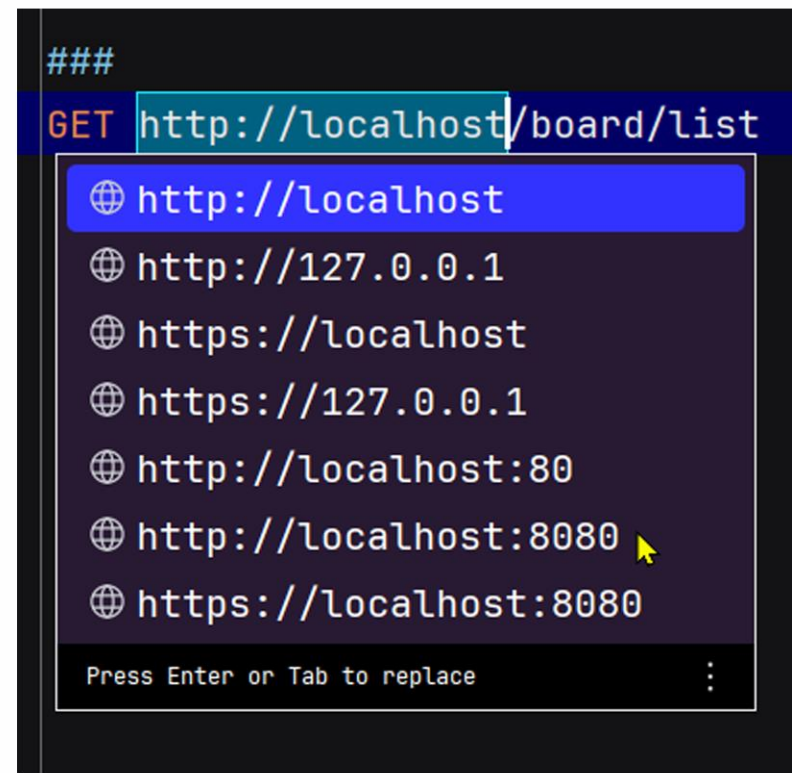
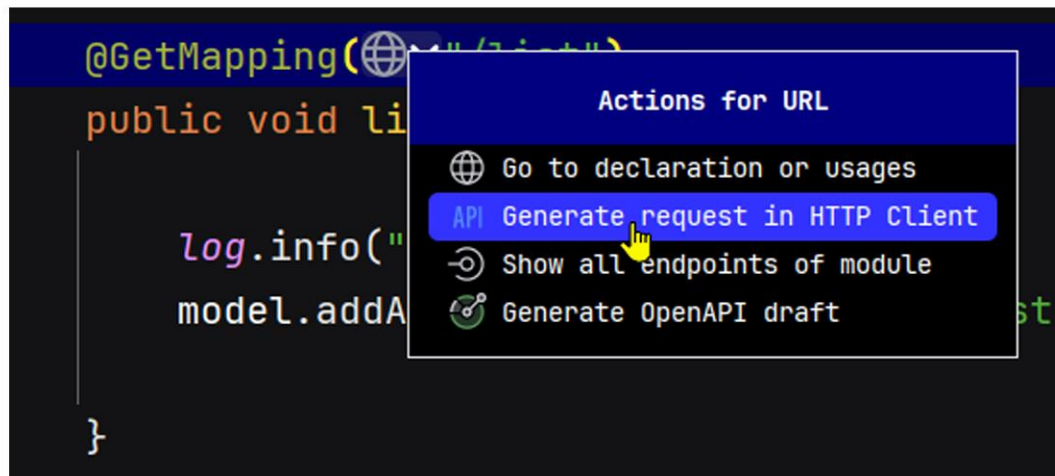
```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
  <head>
    <title>Title</title>
  </head>
  <body>
    ${list}
  </body>
</html>
```


The screenshot shows the IntelliJ IDEA IDE interface. The top menu bar includes File, Edit, View, Navigate, Code, Refactor, Build, Run, Tools, VCS, Window, and Help. The 'Tools' menu is open, displaying various options. The 'HTTP Client' option is highlighted, and its sub-menu is also open, showing options like 'Create Request in HTTP Client', 'Open HTTP Requests Collection', 'Show HTTP Requests History', 'Convert cURL to HTTP Request', 'Add to HTTP Client...', and 'Import from Postman Collection File'. The 'Create Request in HTTP Client' option is selected. The left sidebar shows the project structure with the 'board' folder expanded, revealing the 'src/main/java/org/scoula/board/controller' package. The 'BoardController.java' file is open in the editor, showing the following code:

```
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28
```

The code in the editor includes a package declaration, imports, and the start of the `BoardController` class with a `service` attribute.

✓ Controller에서 바로 생성 가능



The screenshot displays the IntelliJ IDEA IDE interface during the development and testing of the BoardController. The top toolbar includes menus like File, Edit, View, Navigate, Code, Refactor, Build, Run, Tools, VCS, Window, and Help. The breadcrumb at the top indicates the current file is `generated-requests.http` located at `C:\Users\student\AppData\Roaming\JetBrains\IntelliJ\Idea2024.3\scratches\generated-requests.http`.

The Project view on the left shows the directory structure:

- main
 - java
 - org.scoula
 - board
 - controller
 - BoardController
 - domain
 - dto
 - mapper
 - service
 - common-util

The editor window shows the `generated-requests.http` file with the following content:

```
1 #  
2 GET http://localhost:8080/board/list  
3  
4 <> 2025-06-12T171309.200.html
```

The Services view at the bottom left shows the HTTP Request details:

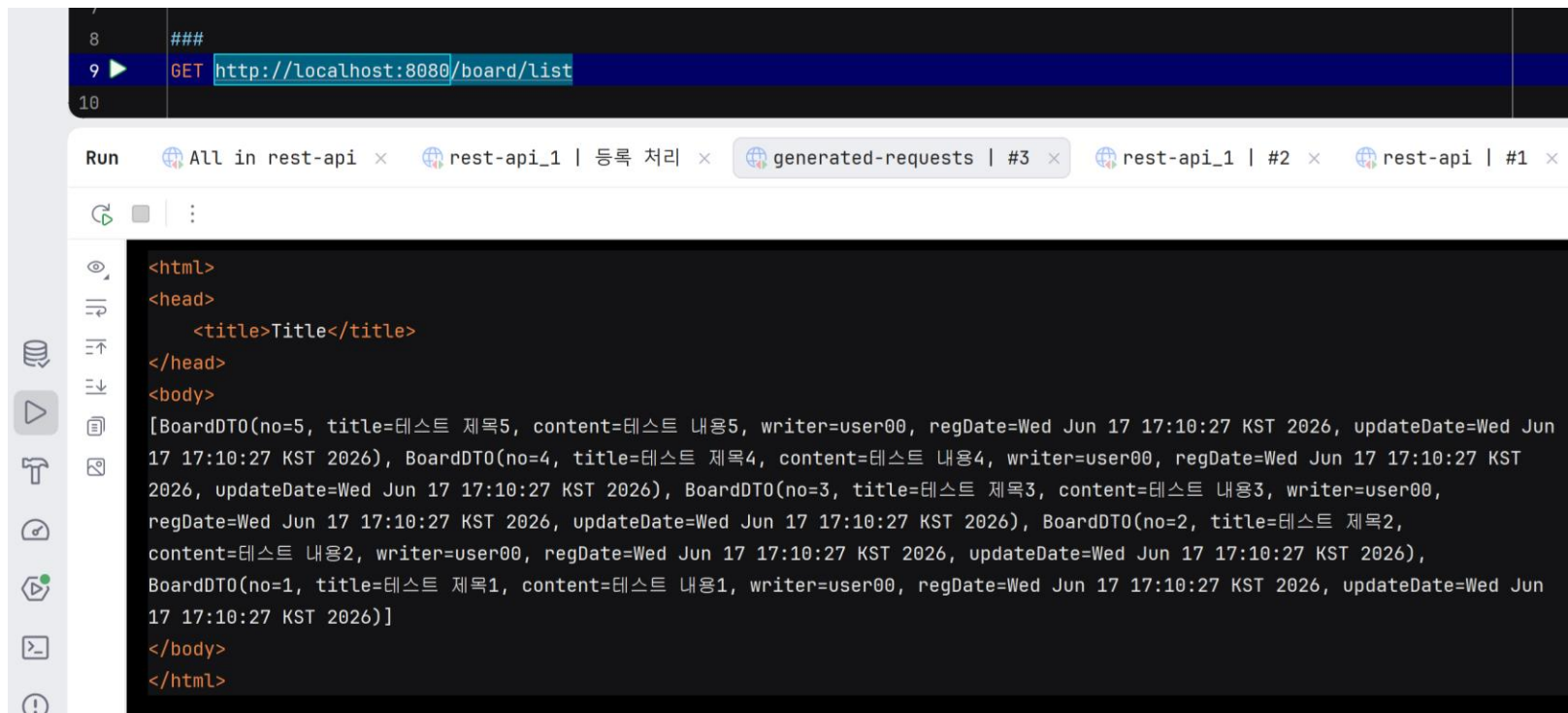
- HTTP Request
 - GET generated-requests | #1 Status: 200 (187 ms)
- Tomcat Server
 - Tomcat 9.0.1051 [local]
 - Gradle : org.scoula : board-1.0-SNAPSHOT.war [Syn]

The bottom right pane shows the response body as HTML:

```
</div> <!-- end of .container -->  
  
<footer class="text-center p-3 footer mt-5">  
  <i class="fa-regular fa-copyright"></i>  
  2023.7 created by java-backend  
</footer>  
</body>  
</html>
```

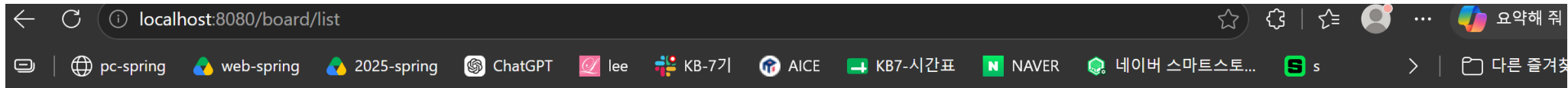
Below the response body, a message states "Response file saved." followed by a link to the response file: `> 2025-06-12T171309.200.html` with a "Show Preview" button.

✓ <http://localhost:8080/board/list>



The screenshot shows a REST client interface. The top bar indicates the request method is GET and the URL is `http://localhost:8080/board/list`. The response is displayed in a dark-themed editor, showing an HTML document with a title and a list of board entries.

```
<html>
<head>
  <title>Title</title>
</head>
<body>
[BoardDTO(no=5, title=테스트 제목5, content=테스트 내용5, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=4, title=테스트 제목4, content=테스트 내용4, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=3, title=테스트 제목3, content=테스트 내용3, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=2, title=테스트 제목2, content=테스트 내용2, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=1, title=테스트 제목1, content=테스트 내용1, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026)]
</body>
</html>
```



```
[BoardDTO(no=5, title=테스트 제목5, content=테스트 내용5, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=4, title=테스트 제목4, content=테스트 내용4, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=3, title=테스트 제목3, content=테스트 내용3, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=2, title=테스트 제목2, content=테스트 내용2, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=1, title=테스트 제목1, content=테스트 내용1, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026)]
```

GET <http://localhost:8080/board/list>

GET <http://localhost:8080/board/create>

POST http://localhost:8080/board/create
Content-Type: application/x-www-form-urlencoded
title=테스트제목&content=테스트내용&writer=apple

GET <http://localhost:8080/board/get?no=1>

POST http://localhost:8080/board/update
Content-Type: application/x-www-form-urlencoded
title=test&content=test&writer=apple2&no=2

✓ 새 글 등록

○ GET 요청

- url: /board/create
- view 이름: board/create

○ POST 요청

- url: /board/create
- 파라미터: BoardDTO
- 처리: service.create()로 실제 등록
- view 이름 : redirect:/board/list

BoardController.java

```
...
@GetMapping("/create") GET :: /board/create
public void create() {
    log.info("create");
}

@PostMapping("/create") POST :: /board/create
public String create(BoardDTO board) {

    log.info("create: " + board);

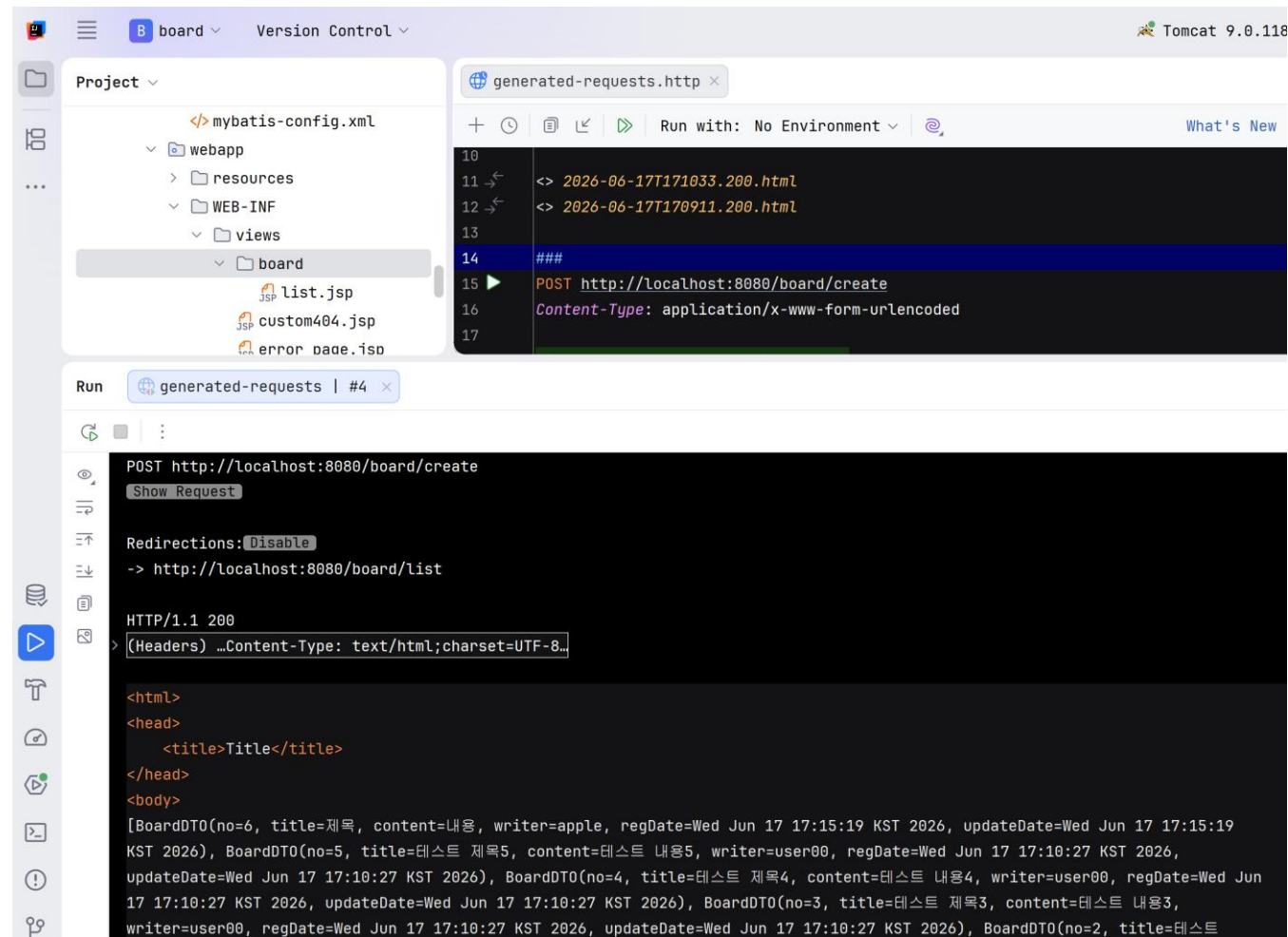
    service.create(board);

    return "redirect:/board/list";
}
```


POST http://localhost:8080/board/create

Content-Type: application/x-www-form-urlencoded

title=제목&content=내용&writer=apple



✓ 글 상세 보기(조회)

○ GET 요청

- url: /board/get
- 파라미터
 - no: 글 번호
- Model 구성
 - 속성명 : board
 - 속성값: service.get()으로 BoardDTO 획득
- View 이름: board/get

○ 수정의 GET 요청 처리와 동일

- url: /board/update

BoardController.java

```
...
@GetMapping({ "/get", "/update" })
public void get(@RequestParam("no") Long no, Model model) {

    log.info("/get or update");
    model.addAttribute("board", service.get(no));
}
```

GET :: /board/get
GET :: /board/update

✓ 글 수정 처리

○ GET 요청

- url: /board/update
- BoardController의 get()에서 처리

○ POST 요청

- url: /board/update
- 파라미터: BoardDTO
- 처리: service.update()로 실제 수정
- view 이름 : redirect:/board/list

BoardController.java

```
@PostMapping("/update") POST :: /board/update
public String update(BoardDTO board) {
    log.info("update:" + board);

    service.update(board);

    return "redirect:/board/list";
}
```

POST http://localhost:8080/board/update

Content-Type: application/x-www-form-urlencoded

no=1&title=kb&content=kb&writer=kb

The screenshot shows an IDE with a project named 'board'. The project structure includes 'config' (RootConfig, ServletConfig, WebConfig), 'controller' (HomeController), 'ex01', and 'exception'. The 'generated-requests.http' file is open, showing a POST request to 'http://localhost:8080/board/update' with content type 'application/x-www-form-urlencoded' and body 'no=1&title=kb&content=kb&writer=kb'. The response is an HTML document with a status of 200. The 'Run' tab shows the response body as a list of BoardDTO objects, with the last object having no=1, title=kb, content=kb, and writer=kb.

```
###
22 POST http://localhost:8080/board/update
23 Content-Type: application/x-www-form-urlencoded
24 no=1&title=kb&content=kb&writer=kb
25
26
27
28
29 <> 2026-06-17T17:25:58.200.html
```

```
[BoardDTO(no=6, title=제목, content=내용, writer=apple, regDate=Wed Jun 17 17:15:19 KST 2026, updateDate=Wed Jun 17 17:15:19 KST 2026), BoardDTO(no=5, title=테스트 제목5, content=테스트 내용5, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=4, title=테스트 제목4, content=테스트 내용4, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=3, title=테스트 제목3, content=테스트 내용3, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=2, title=테스트 제목2, content=테스트 내용2, writer=user00, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:10:27 KST 2026), BoardDTO(no=1, title=kb, content=kb, writer=kb, regDate=Wed Jun 17 17:10:27 KST 2026, updateDate=Wed Jun 17 17:25:58 KST 2026)]
</body>
</html>
```

✓ 삭제 처리

- GET 요청 없음
- POST 요청
 - url: /board/delete
 - 파라미터: no - 삭제할 글 번호
 - 처리: service.delete()로 실제 삭제
 - view 이름 : redirect:/board/list

BoardController.java

```
...  
@PostMapping("/delete") POST :: /board/delete  
public String delete(@RequestParam("no") Long no) {  
  
    log.info("delete..." + no);  
    service.delete(no);  
  
    return "redirect:/board/list";  
}
```

POST http://localhost:8080/board/delete

Content-Type: application/x-www-form-urlencoded

no=1

